Accelute

by Rishi Vhavle aka kaizen

GitHub LinkedIn accelute.co

A case study

How I built an AI tutor that teaches like a teacher

How I designed and built Accelute: verified diagrams, handwritten working and a natural voice, all in sync.

Draws the figure

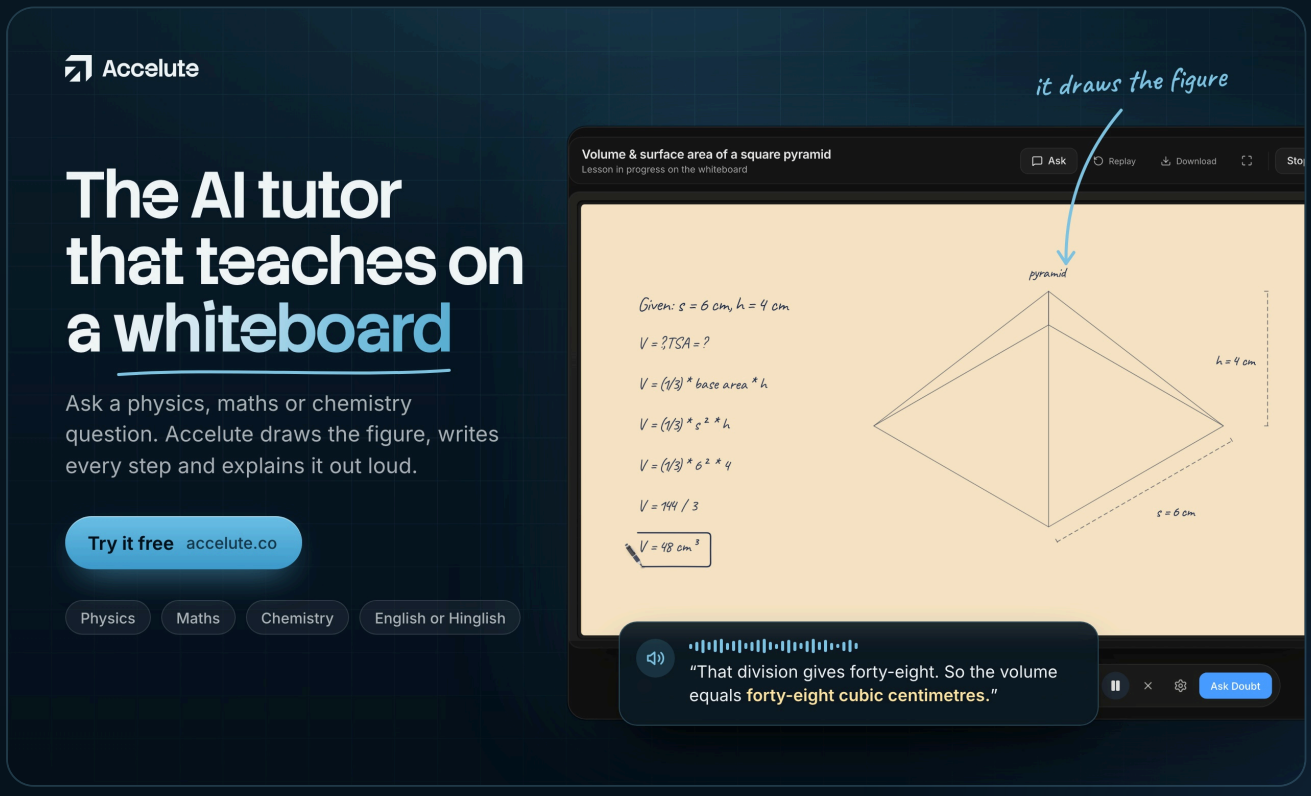
built from the numbers in your question and checked first

Writes every step

by hand, on the board, as the voice speaks it

Explains out loud

in English or Hinglish, at the student's pace



I. The problem

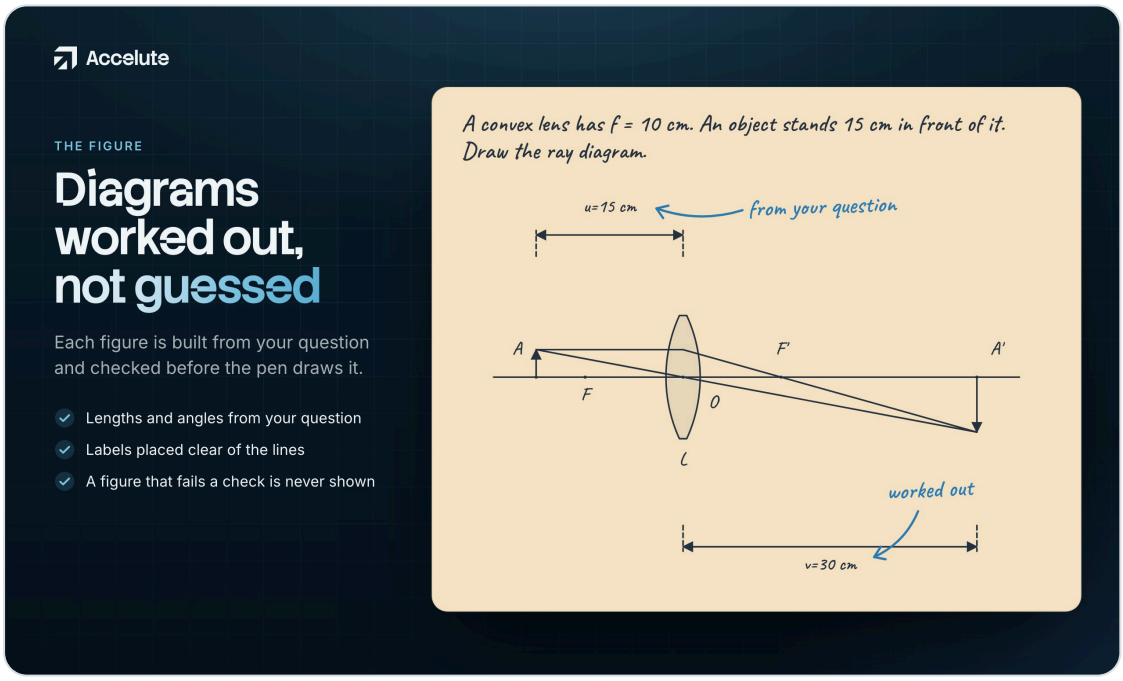
Students learn STEM at a board, not in a chat window

Chat tutors answer STEM questions with walls of text. A projectile question needs a trajectory drawn to scale. A lens question needs rays that actually meet at the image. A LeetCode problem needs the array and the pointers moving step by step. Text cannot do that, and an image model guessing pixels draws figures that look right and are wrong.

A wrong diagram is worse than no diagram, because the student trusts it. So I set one rule on day one:

> *the rule*
>
> Every mark on the board is computed and verified, never guessed. The language model may teach. It may not draw.

That rule made Accelute much harder to build. It is also the reason it is worth using.



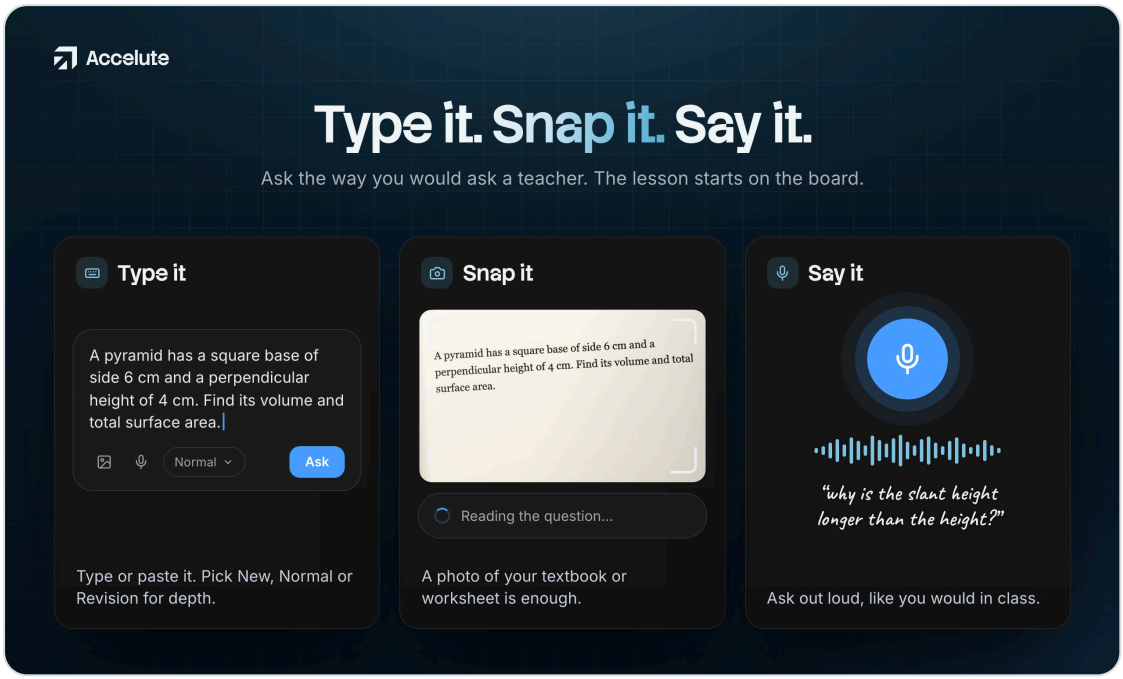
A verified figure. The object distance comes from the question; the image distance is solved, not drawn by eye. A figure that fails any check is never shown.

II. What I built

A lesson that starts on the board within seconds

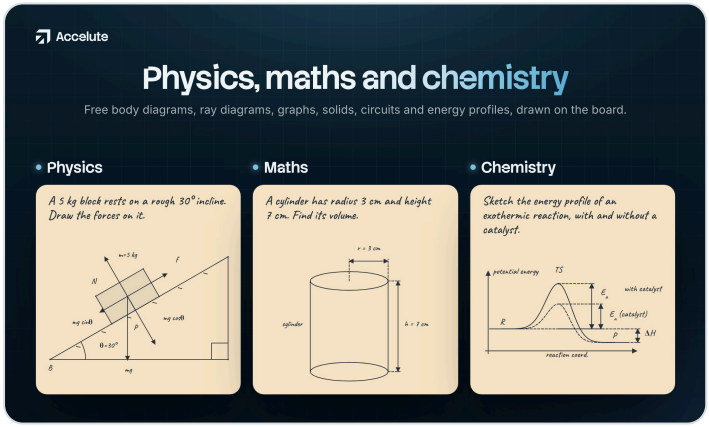
A student types a question, uploads a photo of it, or simply asks out loud. The tutor speaks, the pen writes the working on the left of the board, and a verified diagram reveals itself on the right,

stroke by stroke, timed to the words.



Ask like you would ask a teacher. Typed, photographed or spoken. The familiarity setting (New, Normal, Revision) sets how deep the lesson goes.

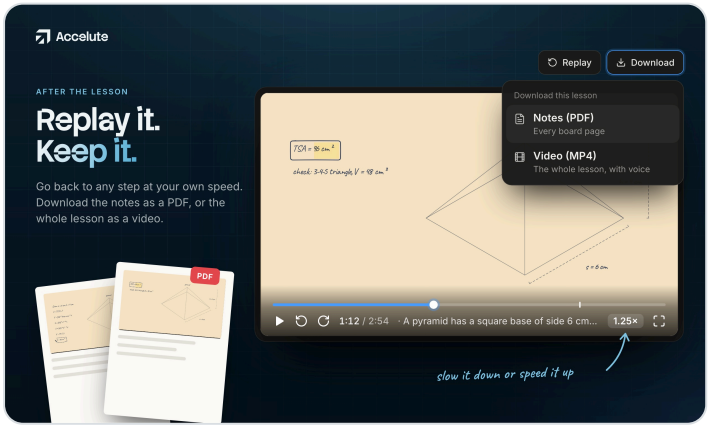
What it does today



Four subject lanes

Maths and physics figures come from a constraint based diagram engine. Chemistry figures are computed from the formula: 13 families, including skeletal organic structures drawn from the compound name.

Programming lessons run a real simulator on the algorithm and draw every board frame from its trace, across 75 LeetCode problem families.



Replay it, keep it

A playback bar with seek, speed and chapters, and a full screen board. Any lesson downloads as PDF notes or as a video with the voice.

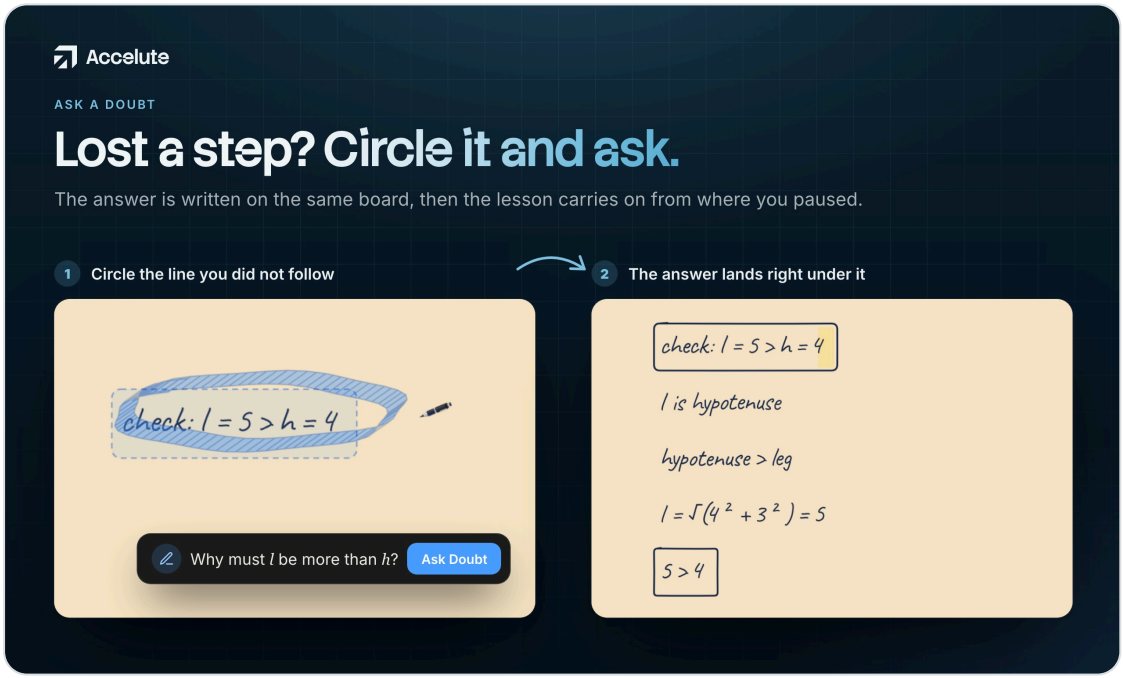
- **Voice and handwriting in sync.** Natural English voices, plus a Hinglish voice for Indian students, with the pen following the spoken words.
- **Everything around the lessons.** Sign in with Google or email, saved lessons with private storage for audio and photos, a landing site, and an admin view where I watch usage and read every failed lesson.

III. Asking doubts

One lesson that keeps going until it makes sense

In a real class, the most useful moment is when a student stops the teacher and says, wait, I did not get that line. I wanted Accelute to work the same way.

At any point in a lesson, a student can circle the line they lost right on the board, or just type or say what confused them, and press Ask Doubt. The lesson pauses. The tutor does not start over or open a new page. It thinks right there on the same board and writes the answer under what is already there, with the same figure and the same numbers, the way a teacher would.



Circle it and ask. The student rings the step they did not follow; the answer is written right under it, on the same page.

When a student circles a line, the tutor knows exactly which line they mean. Everything on the board is stored with its exact text and position, so the circle is matched to that step instead of being guessed from a picture. If the circle lands on an empty part of the board, the tutor says so rather than pretending.

Once the doubt is answered, the student decides what happens next: continue the lesson from where it paused, or ask another doubt. The tutor never carries on by itself or talks over them. They can ask a doubt about the answer to a doubt, and keep going like that for as long as they need.

When the page fills up, the board moves to a fresh page and keeps the figure, so the whole conversation stays inside one lesson.

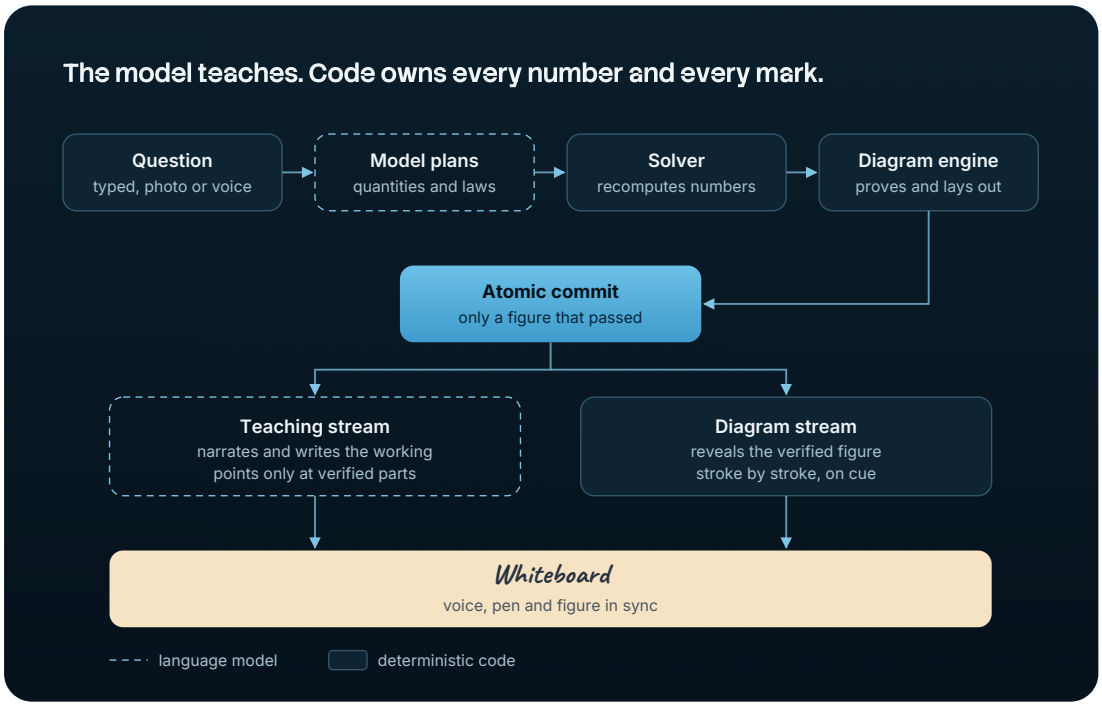
It all stays together afterwards. The lesson, the doubts and the answers are saved on the same board, so a replay or a downloaded copy shows exactly how the student got there. If they close the tab halfway, they can come back later and continue from the same spot.

> *why it matters*
>
> A student does not have to understand everything the first time. A lecture becomes a conversation, and they can keep asking until it makes sense.

IV. My contribution

Two streams, one authority boundary

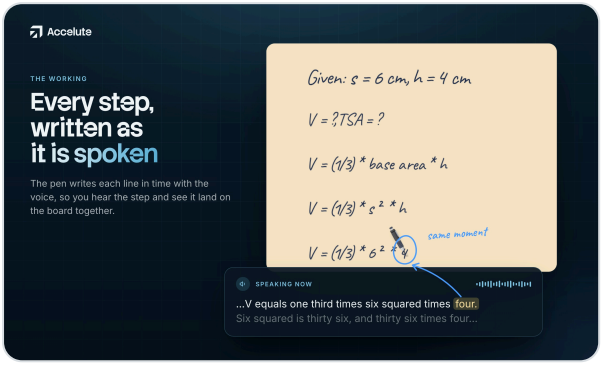
I started building Accelute in June 2026 and I have built every part of it myself: the diagram engine, the lesson pipeline, the voice and handwriting sync, the web app, the landing site and the servers it runs on.



The decision that shaped everything was the architecture. The diagram stream is deterministic code. The teaching stream is the language model. The model can narrate, write equations in the work area and point at verified parts of the figure. It can never draw, label or supply a coordinate, and the runtime enforces that.

Every lesson runs through the same pipeline. The model plans the question and lists the quantities and laws involved. A deterministic solver recomputes every number from the question itself, so a wrong number from the model can never sit beside a correct figure. The engine compiles

candidate figures, validates and proves them, lays out the labels, and picks the most honest tier that passed: exact, qualitative, or a faithful picture of the question. Only then does the figure commit and the narration start. If nothing passes, the board stays clean and the lesson still teaches.



Voice and pen in sync

The digit lands on the board at the moment the voice says it. Getting there took six separate fixes, each one measured first and then guarded by a check.

The hardest problems I solved

Each one started with a measurement, and each fix ended with an automated gate that keeps it fixed.

Problem	What I did	Result
Silent gaps between spoken sentences	Generated upcoming sentences ahead on named voice streams, and closed each stream properly so its end arrived on time	**1,211 ms → 20 ms** median gap
Slow lesson start	Profiled the turn end to end. The step that grounds the numbers was timing out, so I rebuilt how it runs	**43 s → 25 s** ask to first word, median
Pen out of step with the voice	Found six separate causes (playback rate, unused exact timings, wrong writing speed, missed word matches and more) and fixed each with its own test	**Pen follows the spoken word**
Jerky handwriting	Found that chained animations each lost a frame: 33 ms of every 50 ms glyph was dead time	**44x → 10x** speed spread in one figure
Leader lines crossing out their labels	Rewrote how the label engine routes leaders, then counted collisions across the whole corpus	**893 → 0** collisions
Voice misreading notation	Built a round trip check: speak each line, transcribe it back, compare the words	**2,567 → 6** hazardous lines
Costly, uneven model calls	Added a provider switch and ran the same harness on each model	**About a third of the cost** on sponsored cloud credits

V. How I work

I test it the way a student would use it

Most of my time does not go into new features. It goes into watching lessons. I built a small tool that plays a full lesson from start to finish, the same way a student would, and saves a picture of the board at every step. That way I look at what a student actually sees, instead of guessing from the code. When I change something, I try it on questions I did not use while building it, so I know it works in general and not only on my own examples.

When I find a problem, I try to fix it once. Every fix comes with an automatic check that fails if the problem ever comes back, and now and then I break the code on purpose to make sure the check really catches it. It feels slower in the moment, but it lets me keep changing things without quietly breaking something I fixed a month ago.

I have also learned to let go of my own work. Early on I drew diagrams by hand for single topics. They looked great in a demo, but every new topic needed more hand work, so I stopped and built general pieces that work across many questions instead. Some changes made no real difference when I tested them, and I left those out.

I build for the students I know. Many of them think in a mix of Hindi and English, so the tutor can explain in Hinglish while the board stays in English, the way their exams are written. And because I run everything myself, from the server to every release, I am also the one who notices when something goes wrong and puts it right.

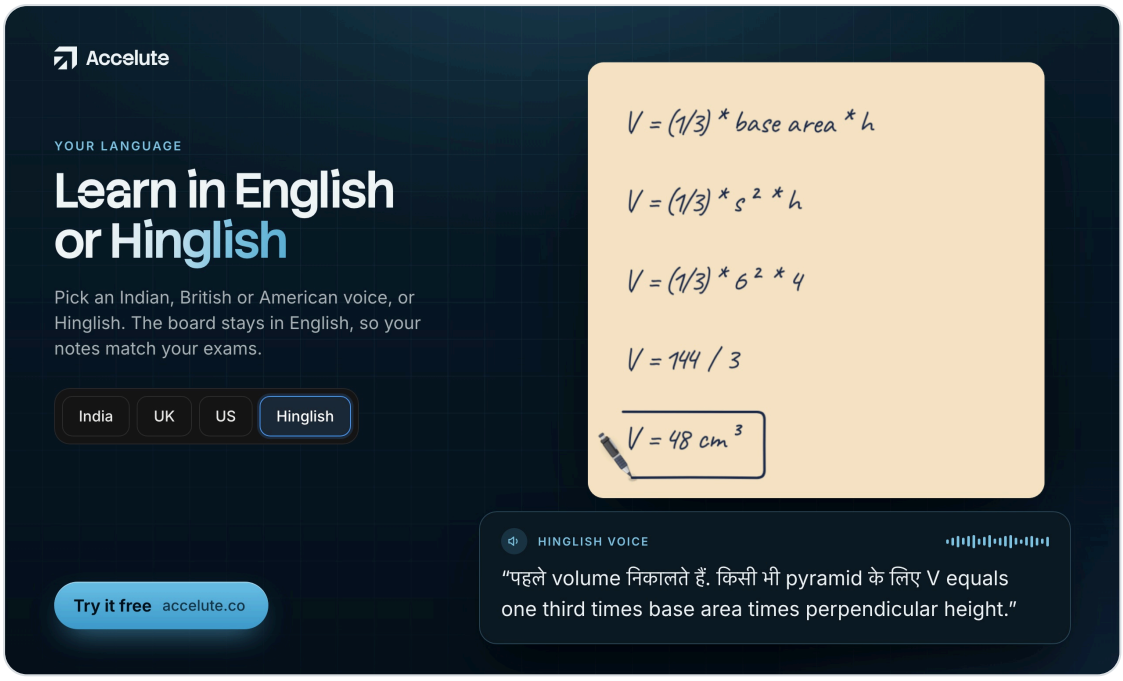
VI. What I learned

What building Accelute taught me

The biggest thing I learned was to be clear about what the AI should and should not do. It is good at explaining things in a friendly, natural way, but it is not reliable at drawing exact figures. So I let it do the talking and kept the drawing in code that I can check. That one decision made the whole thing much easier to trust.

I also learned to measure a problem before trying to fix it. Small things, like a short pause in the voice or the pen running a little behind the words, are easy to ignore. Once I actually measured them, they were much easier to fix, and they stayed fixed.

There is still plenty to improve. Some figures still miss parts of what the question asks for, and that is what I am working on now. I also want more students to use Accelute, so that their real questions guide what I build next.

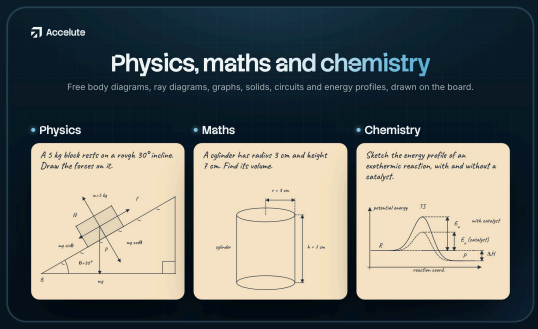


Built for the students I know. A Hinglish voice for Indian students, while the board stays in English so the notes match their exams.

see it for yourself

Try a lesson at accelute.co

Ask any maths, physics, chemistry or programming question and watch the board fill in, one stroke at a time.



by Rishi Vhavle aka kaizen

GitHub LinkedIn accelute.co